LAB5 MCU1
실험 후 보고서

실험자 : 권진혁
학번 : 2023440007
학과 : 전자전기컴퓨공학부
실험 일시 : 2026. 10. 19.

1. FPGA 실습 - Piano

· 설계 목표

피아노(STM32_1) 실습의 목표는 FPGA를 이용하여 여러 개의 버튼 입력에 따라 서로 다른 음계(도, 레, 미, 파, 솔, 라, 시, 도)를 선택하고, 선택된 음에 해당하는 주파수의 사각파(square wave)를 생성하여 피에조 스피커로 출력하는 디지털 음원 발생기를 구현하는 것이다. 또한 FPGA의 클록 분주 개념과 순차회로 설계 방법을 익히고, 버튼 입력에 따른 음계 선택, 주파수 생성, 무음 처리, 리셋 기능 등을 하나의 시스템으로 통합하여 동작하도록 설계하는 것을 목표로 한다.

· 핵심 코드

```
`timescale 1ns/1ps
// 버튼 입력에 대응하는 음의 주파수를 선택합니다.
// 보드의 버튼 입력은 안정된 HIGH 활성 신호로 전달합니다.
module note_select(input wire [7:0] key, output reg [31:0] frequency_hz);
    always @* begin
        case(key)
            8'h80: frequency_hz=262;
            8'h40: frequency_hz=294;
            8'h20: frequency_hz=330;
            8'h10: frequency_hz=349;
            8'h08: frequency_hz=392;
            8'h04: frequency_hz=440;
            8'h02: frequency_hz=494;
            8'h01: frequency_hz=523;
            default: frequency_hz=0; // 무입력이나 여러 버튼 입력은 무음
        endcase
    end
endmodule
```

note_select 모듈은 사용자가 누른 버튼 값을 해석하여 해당 음계의 주파수를 출력하는 조합회로이다. 8비트 입력 key를 받아 각 비트에 대응되는 음계의 주파수(262Hz~523Hz)를 frequency_hz로 출력한다. 예를 들어 8'h80이 입력되면 도(C4, 262Hz), 8'h40이면 레(D4, 294Hz)가 선택된다. 또한 여러 키가 동시에 눌리거나 아무 키도 입력되지 않은 경우에는 기본값으로 0Hz를 출력하여 무음 상태를 만든다. 이를 통해 하나의 버튼 입력만 허용하고 잘못된 입력 상황에서는 스피커가 동작하지 않도록 설계하였다.

```
`timescale 1ns/1ps
module square_tone #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz, rst_p,
    input wire [31:0] frequency_hz,
    output reg tone
);
    reg [31:0] count, previous_hz;
    wire [31:0] half_cycles = frequency_hz == 0 ? 1 :
                              CLK_HZ / (2 * frequency_hz);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin count<=0; tone<=0; previous_hz<=0; end
        else if(frequency_hz==0 || frequency_hz!=previous_hz) begin
            count<=0; tone<=0; previous_hz<=frequency_hz;
        end
        else if(count >= half_cycles-1) begin count<=0; tone<=~tone; end
        else count<=count+1;
    end
endmodule
```

square_tone 모듈은 선택된 주파수 정보를 받아 실제 오디오 신호를 생성하는 순차회로이다. 입력된 주파수에 대해 반주기(half_cycles)를 계산한 후, FPGA 클록 카운터가 해당 값에 도달할 때마다 출력 신호 tone을 반전시킨다. 결과적으로 출력은 HIGH와 LOW가 반복되는 사각파가 되며, 이 사각파의 주파수가 사용자가 선택한 음계와 일치하게 된다. 또한 음계가 변경되거나 주파수가 0이 되면 카운터와 출력을 초기화하여 음 전환 시 발생할 수 있는 잡음을 줄이고 안정적으로 새로운 음을 출력하도록 구현하였다.

```
`timescale 1ns/1ps
module piano #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz, rst_p,
    input wire [7:0] key,
    output wire piezo
);
    wire [31:0] frequency_hz;
    note_select u_note(key, frequency_hz);
    square_tone #(.CLK_HZ(CLK_HZ)) u_tone(clk_50mhz,rst_p,frequency_hz,piezo);
endmodule
```

piano 모듈은 최상위 모듈로서 음계 선택 모듈과 사각파 생성 모듈을 연결하는 역할을 한다. 버튼 입력 key는 note_select에서 주파수 값으로 변환되고, 변환된 주파수는 square_tone으로 전달되어 최종적으로 piezo 출력 신호를 생성한다. 즉, 이 모듈은 사용자의 입력부터 스피커 출력까지의 전체 데이터 흐름을 관리하며, FPGA 기반 전자 피아노의 전체 기능을 하나의 회로로 구성하

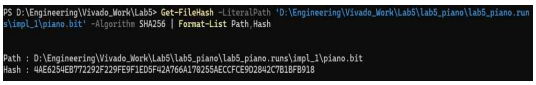
는 역할을 한다. 상위 설계에서는 모듈 분할을 통해 기능별 설계와 유지보수가 용이하도록 구조화하였다.

```
`timescale 1ns/1ps
module tb;
    reg clk=0, rst=1;
    always #5 clk=~clk;
    reg [7:0] key=0;
    wire piezo;
    reg [7:0] select_key;
    wire [31:0] frequency_hz;
    note_select select_dut(select_key,frequency_hz);
    piano #(.CLK_HZ(10000)) dut(clk,rst,key,piezo);
    integer notes[0:7];
    integer n, j, half_cycles;
    reg expected;
    task step; begin @(posedge clk); #1; end endtask
    initial begin
        notes[0]=262; notes[1]=294; notes[2]=330; notes[3]=349;
        notes[4]=392; notes[5]=440; notes[6]=494; notes[7]=523;
        step; @(negedge clk); rst=0;
        for(n=0;n<8;n=n+1) begin
            select_key=8'h80 >> n; #1;
            if(frequency_hz !== notes[n]) $fatal(1,"note lookup");
            @(negedge clk); key=8'h80 >> n; step;
            if(piezo !== 0) $fatal(1,"note change reset");
            half_cycles=10000/(2*notes[n]);
            expected=0;
            for(j=1;j<=4*half_cycles;j=j+1) begin
                step;
                if(j%half_cycles==0) expected=~expected;
                if(piezo !== expected) $fatal(1,"tone timing note=%0d",n);
            end
        end
        @(negedge clk); key=8'hC0; select_key=8'hC0; step;
        if(piezo !== 0 || frequency_hz !== 0) $fatal(1,"multiple keys");
        @(negedge clk); key=0; select_key=0;
        repeat(40) begin step; if(piezo !== 0) $fatal(1,"silence"); end
        @(negedge clk); rst=1; step;
        if(piezo !== 0) $fatal(1,"reset");
        $display("PASS piano: all 8 tones, silence, multiple keys, reset"); $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

TB는 피아노 시스템이 설계 의도에 맞게 동작하는지 자동으로 검증하기 위해 작성되었다. TB에서는 FPGA의 시스템 클록과 리셋 신호를 생성한 후, 각 버튼 입력을 순차적으로 인가하여 도(C4), 레(D4), 미(E4), 파(F4), 솔(G4), 라(A4), 시(B4), 높은 도(C5)에 해당하는 주파수가 정상적으로 선택되는지 확인하였다. 또한 선택된 주파수에 따라 출력 사각파의 토글 주기가 올바르게 생성되는지 측정하였으며, 아무 버튼도 누르지 않은 경우와 여러 버튼을 동시에 누른 경우에는 무음 상태가 유지되는지 검증하였다. 마지막으로 리셋 신호 입력 시 카운터와 출력 신호가 정상적으로 초기화되는지 확인하여 전체 피아노 시스템의 기능, 예외 상황 처리, 안정성을 종합적으로 검증하였다.

· **시뮬레이션 결과**

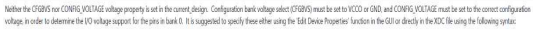
- Bit Hash



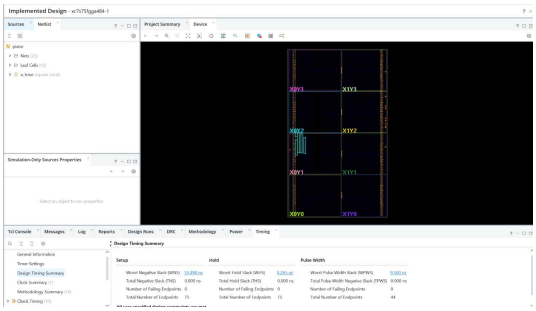
- Bitstream



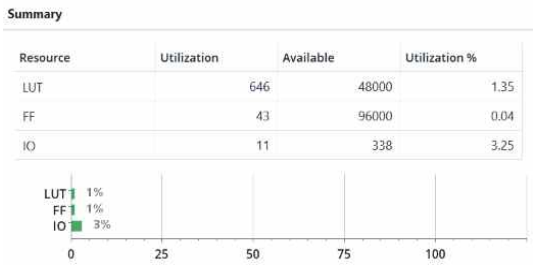
- DRC



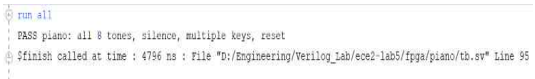
- Timing



- Utilization



- Vivado Pass



- Simulation



· **검증 결과**

TB에서는 8개의 음계에 대해 각각 올바른 주파수가 선택되는지 확인하였으며, 각 음의 반주기에 맞추어 사각파가 정확히 생성되는지도 검증하였다. 또한 여러 버튼을 동시에 누른 경우 무음 상태가 유지되는지, 버튼이 입력되지 않은 경우 스피커 출력이 발생하지 않는지, 그리고 리셋 신호가 입력되었을 때 출력이 정상적으로 초기화되는지를 확인하였다. 모든 검증 항목을 통과하였으며 최종적으로 "PASS piano: all 8 tones, silence, multiple keys, reset" 메시지가 출력되어 설계가 의도한 대로 동작함을 확인하였다.

· **문제 해결 과정**

설계 과정에서 가장 중요한 문제는 FPGA의 50MHz 클록을 사람이 들을 수 있는 수백 Hz 영역의 음계 신호로 변환하는 것이었다. 이를 해결하기 위해 원하는 음의 주파수로부터 반주기 값을 계산한 후 카운터를 이용해 출력 신호를 주기적으로 반전시키는 방식의 주파수 분주기를 구현하였다. 또한 음이 변경될 때 카운터 값이 그대로 유지되면 출력 파형이 불안정해질 수 있으므로 주파수 변경 감지 기능을 추가하여 카운터와 출력을 초기화하였다. 마지막으로 여러 버튼이 동시에 눌리는 경우 예상치 못한 음이 생성되는 문제를 방지하기 위해 해당 상황에서는 주파수를 0Hz로 설정하여 무음 처리함으로써 안정적인 피아노 동작을 구현하였다.

2. FPGA 실습 - 신호등 제어

· **설계 목표**

신호등 제어 실습의 목표는 FPGA를 이용하여 실제 교차로의 신호 체계를 모의 구현하는 것이다. 50MHz 시스템 클록을 이용해 1초 주기의 기준 신호를 생성하고, 네 방향의 차량 신호와 보행자 신호를 순차적으로 제어하도록 설계하였다. 차량 신호는 초록불(11초)과 노란불(2초)을 거쳐 다음 방향으로 전환되며, 보행자 신호는 일정 시간 동안 녹색 점등 후 점멸하도록 구현하였다. 또한 비상 상황을 가정한 점멸 모드(flicker)와 리셋 기능을 포함하여 실제 신호등 제어 시스템의 동작 원리를 FPGA 기반 순차회로로 학습하는 것을 목표로 한다.

· **핵심 코드**

```
`timescale 1ns/1ps
// 느린 동작은 새 클록 대신 한 클록 길이의 tick으로 만듭니다.
module second_tick #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz, rst_p,
    output wire tick,
    output wire blink
);
    reg [31:0] count;
    assign tick=(count==CLK_HZ-1);
    assign blink=(count < CLK_HZ/2);
    always @(posedge clk_50mhz)
        if(rst_p || tick) count<=0;
        else count<=count+1;
endmodule
```

second_tick 모듈은 50MHz 클록을 이용하여 1초에 한 번 발생하는 tick 신호와 점멸 제어용 blink 신호를 생성한다. 내부 카운터가 CLK_HZ-1까지 증가하면 tick을 발생시키고 다시 0으로 초기화된다. 또한 카운터가 절반 이하일 때 blink=1, 그 이후에는 blink=0이 되므로 0.5초 주기의 점멸 효과를 얻을 수 있다. 이 모듈은 신호등 전체 시스템에서 시간을 관리하는 기준 타이머 역할을 수행하며, 모든 상태 전환은 이 tick 신호를 기준으로 이루어진다.

```
`timescale 1ns/1ps
// 각 방향에서 0~10초는 초록, 11~12초는 노랑입니다.
module traffic_sequence(
    input wire clk_50mhz,rst_p,tick,flicker,
    output reg [1:0] direction,
    output reg [3:0] phase
);
    always @(posedge clk_50mhz) begin
        if(rst_p || flicker) begin direction<=0; phase<=0; end
        else if(tick)
            if(phase==12) begin phase<=0; direction<=direction+1'b1; end
            else phase<=phase+1'b1;
    end
endmodule
```

traffic_sequence 모듈은 현재 방향(direction)과 현재 단계(phase)를 관리하는 상태 제어 회로이다. 매초 tick 신호가 발생할 때마다 phase가 증가하며, phase=12에 도달하면 0으로 초기화되고 다음 방향으로 이동한다. 따라서 각 방향은 총 13단계(0~12)를 가지며, 0~10초는 차량 초록불, 11~12초는 노란불 상태가 유지된다. 또한 flicker 또는 reset이 입력되면 방향과 단계를 모두 초기 상태로 되돌려 비상 점멸 모드에 즉시 진입할 수 있도록 설계되었다.

```
`timescale 1ns/1ps
module traffic_lights(
    input wire [1:0] direction,
    input wire [3:0] phase,
    input wire flicker,blink,
    output reg [3:0] red,yellow,green,walk_red,walk_green
);
    reg [3:0] vehicle_mask,pedestrian_mask;
    always @* begin
        // 방향별 차량과 보행자 LED를 선택합니다.
        case(direction)
            0: begin vehicle_mask=4'b1000; pedestrian_mask=4'b0100; end
            1: begin vehicle_mask=4'b0100; pedestrian_mask=4'b0010; end
            2: begin vehicle_mask=4'b0010; pedestrian_mask=4'b0001; end
            3: begin vehicle_mask=4'b0001; pedestrian_mask=4'b1000; end
        endcase
        red=4'b1111; yellow=0; green=0; walk_red=4'b1111; walk_green=0;
        if(flicker) begin
            red=0; yellow={4{blink}}; walk_red={4{blink}};
        end else begin
            if(phase<=10) begin green=vehicle_mask; red=~vehicle_mask; end
            else begin yellow=vehicle_mask; red=~vehicle_mask; end
            if(phase<=3) begin walk_green=pedestrian_mask; walk_red=~pedestrian_mask; end
            else if(phase<=10) begin
                walk_green=blink ? pedestrian_mask : 4'b0000;
                walk_red=~pedestrian_mask;
            end
        end
    end
endmodule
```

traffic_lights 모듈은 현재 방향과 단계 정보를 실제 LED 출력 신호로 변환한다. 차량 신호등은 선택된 방향에 대해서만 초록색 또는 노란색이 켜지고, 나머지 방향은 빨간색이 유지된다. 보행자 신호는 단계 0~3 동안 녹색이 점등되고, 단계 4~10에서는 blink 신호에 따라 점멸하도록 구현되었다. 또한 flicker 입력이 활성화되면 정상적인 신호 제어를 중단하고 모든 방향의 노란불과 보행자 적색등을 동시에 점멸시켜 비상 운전 모드를 구현한다. 이를 통해 실제 교통 신호 체계와 유사한 동작을 재현할 수 있다.

```
`timescale 1ns/1ps
module traffic #(parameter CLK_HZ=50_000_000)(
    input wire clk_50mhz,rst_p,flicker,
    output wire [3:0] led_red,led_yellow,led_green,led_left,
    output wire [3:0] led_walk_red,led_walk_green
);
    wire tick,blink;
    wire [1:0] direction;
    wire [3:0] phase;
    second_tick #(.CLK_HZ(CLK_HZ)) u_tick(clk_50mhz,rst_p,tick,blink);
    traffic_sequence u_sequence(clk_50mhz,rst_p,tick,flicker,direction,phase);
    traffic_lights u_lights(direction,phase,flicker,blink,
        led_red,led_yellow,led_green,led_walk_red,led_walk_green);
    assign led_left=led_green;
endmodule
```

최상위 연결 모듈은 신호등 제어 시스템의 모든 하위 모듈을 통합하는 역할을 수행한다. 외부로부터 입력되는 50MHz 시스템 클록(clk), 리셋 신호(reset), 비상 점멸 신호(flicker)를 받아 시간 생성 모듈(second_tick), 상태 제어 모듈(traffic_sequence), 출력 제어 모듈(traffic_lights)을 서로 연결한다. 먼저 second_tick에서 생성된 1초 주기의 tick 신호와 blink 신호가 traffic_sequence와 traffic_lights로 전달된다. traffic_sequence는 현재 방향(direction)과 단계(phase)를 계산하고, 이 정보를 traffic_lights에 제공한다. 마지막으로 traffic_lights는 해당 정보를 이용하여 차량 신호등 LED(빨강, 노랑, 초록), 보행자 신호등 LED, 좌회전 표시 LED를 출력한다. 즉, 최상위 모듈은 각 기능 모듈 간의 데이터 흐름을 관리하며 사용자의 입력이 실제 신호등 출력으로 이어지는 전체 시스템의 중심 역할을 수행한다.

```
`timescale 1ns/1ps
module tb;
    reg clk=0,rst=1,flicker=0;
    always #5 clk=~clk;
    wire [3:0] red,yellow,green,left,wr,wg;
    traffic #(.CLK_HZ(4)) dut(clk,rst,flicker,red,yellow,green,left,wr,wg);
    integer direction,phase,subcycle;
    reg [3:0] vm,pm,er,ey,eg,ewr,ewg;
    task step; begin @(posedge clk); #1; end endtask
    task check;
        begin
            case(direction)
                0: begin vm=8; pm=4; end
                1: begin vm=4; pm=2; end
                2: begin vm=2; pm=1; end
                3: begin vm=1; pm=8; end
            endcase
            er=~vm; ey=0; eg=0; ewr=15; ewg=0;
            if(phase<11) eg=vm; else ey=vm;
            if(phase<4) begin ewg=pm; ewr=~pm; end
            else if(phase<11) begin
                ewr=~pm; if(subcycle<2) ewg=pm;
            end
            if({red,yellow,green,wr,wg} !== {er,ey,eg,ewr,ewg})
                $fatal(1,"traffic dir=%0d phase=%0d sub=%0d",direction,phase,subcycle);
            if(left !== green) $fatal(1,"left indicator");
        end
    endtask
    initial begin
        step; @(negedge clk); rst=0;
        // 네 방향의 13단계와 각 초의 점멸을 확인합니다.
        for(direction=0;direction<4;direction=direction+1)
            for(phase=0;phase<13;phase=phase+1)
                for(subcycle=0;subcycle<4;subcycle=subcycle+1) begin
                    check; step;
                end
        direction=0; phase=0; subcycle=0; check;
        @(negedge clk); flicker=1; step;
        repeat(8) begin
            if(red !== 0 || green !== 0 || wg !== 0) $fatal(1,"flicker outputs");
            if(yellow !== wr || !(yellow===0 || yellow===15)) $fatal(1,"flicker blink");
            step;
        end
        @(negedge clk); flicker=0; rst=1; step;
        direction=0; phase=0; subcycle=0; check;
        $display("PASS traffic: full cycle, pedestrian blink, flicker, reset"); $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

TB는 실제 FPGA 보드 없이도 설계된 신호등 제어기가 정상적으로 동작하는지 검증하기 위해 작성되었다. 시뮬레이션 시간을 줄이기 위해 시스템 클록 주파수를 매우 작은 값으로 설정하고, 클록 생성 블록을 이용해 주기적인 클록 신호를 발생시킨다. 이후 리셋 신호를 인가하여 시스템을 초기화한 후, 네 방향의 차량 신호가 순차적으로 초록불과 노란불 상태를 거쳐 다음 방향으로 전환되는지를 검사한다. 또한 보행자 신호가 녹색 점등 후 정상적으로 점멸하는지 확인하고, 비상 점멸 신호(flicker)를 활성화하여 모든 신호등이 노란색 점멸 상태로 전환되는지도 검증한다. 마지막으로 리셋을 다시 인가하여 상태 변수와 출력이 초기 상태로 복원되는지 확인하며, 모든 검증이 통과하면 PASS 메시지를 출력하도록 구성되어 있다. 이를 통해 정상 동작, 예외 처리, 비상 모드, 리셋 기능까지 설계 전반의 신뢰성을 확인할 수 있다.

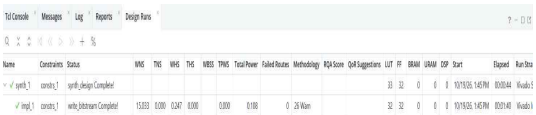
· **시뮬레이션 자료**

- Bit Hash

```
PS D:\Engineering\Vivado_Work\Lab5> Get-FileHash -LiteralPath 'D:\Engineering\Vivado_Work\Lab5\lab5_traffic\lab5_traffic
.runs\impl_1\traffic.bit' -Algorithm SHA256 | Format-List Path,Hash


Path : D:\Engineering\Vivado_Work\Lab5\lab5_traffic\lab5_traffic.runs\impl_1\traffic.bit
Hash : F1C3571E5304164967144182EBAF08FB22B6A0367607D98D2EE7804643A83F92B92
```

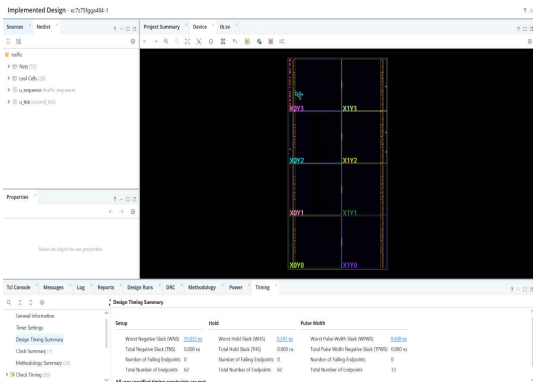
- Bitstream



- DRC

Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax:

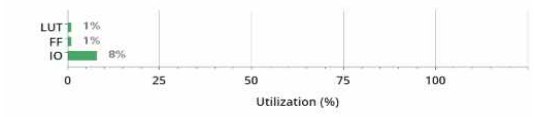
- Timing



- Utilization

Summary

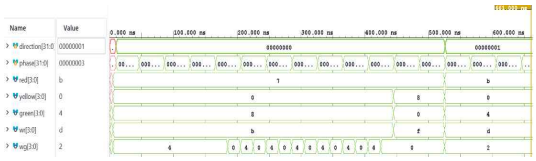
Resource	Utilization	Available	Utilization %
LUT	32	48000	0.07
FF	32	96000	0.03
IO	27	338	7.99



- Vivado Pass

```
run all
PASS traffic: full cycle, pedestrian blink, flicker, reset
$finish called at time : 2186 ns : File "D:/Engineering/Verilog_Lab/ece2-lab5/fpga/traffic/tb.sv" Line 107
```

- Simulation



· **검증 결과**

TB에서는 네 방향의 교차로에 대해 각 방향별 13단계의 상태 변화를 모두 확인하였다. 차량 신호의 초록불과 노란불 전환, 보행자 녹색 신호 및 점멸 동작, 좌회전 표시 LED 동기화 여부를 검증하였으며, 비상 점멸 모드(flicker) 진입 시 모든 출력이 설계 조건에 맞게 점멸하는지도 확인하였다. 또한 리셋 입력 후 방향과 단계가 초기 상태로 복원되는지를 검사하였다. 모든 검증이 성공적으로 완료되었으며 최종적으로 "PASS traffic: full cycle, pedestrian blink, flicker, reset" 메시지가 출력되어 설계가 정상 동작함을 확인하였다.

· **문제 해결 과정**

신호등 설계 과정에서 가장 중요한 문제는 FPGA의 고속 클록을 실제 신호등에서 사용하는 초 단위 시간으로 변환하는 것이었다. 이를 해결하기 위해 클록 분주 대신 tick 신호를 이용한 동기식 상태 제어 방식을 적용하였다. 또한 보행자 신호의 점멸 기능 구현을 위해 별도의 blink 신호를 생성하여 녹색 LED 출력에 적용하였다. 비상 점멸 모드에서는 기존 상태를 모두 무시하고 즉시 점멸 상태로 전환해야 하므로 flicker 입력 시 상태를 초기화하고 별도의 출력 로직을 적용하였다. 이 과정을 통해 차량 신호, 보행자 신호, 점멸 모드가 서로 충돌하지 않고 안정적으로 동작하는 신호등 제어 시스템을 구현할 수 있었다.

3. FPGA 실습 - ADC

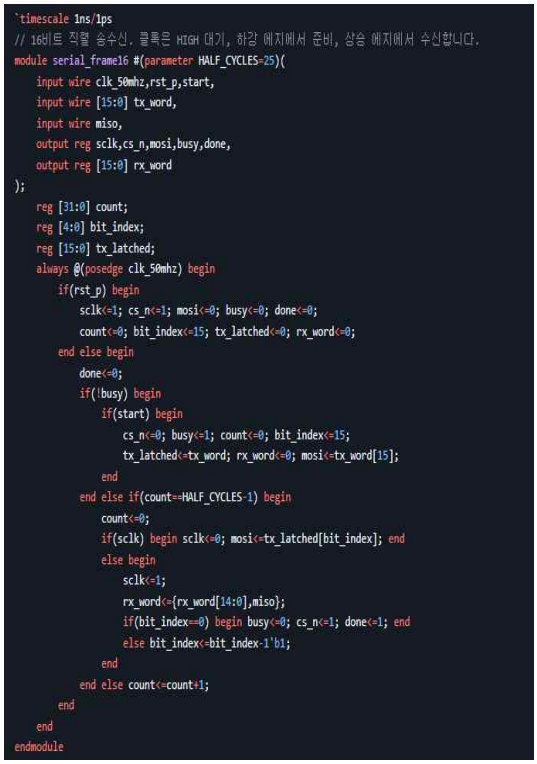
· **설계 목표**

FPGA와 ADC(Analog-to-Digital Converter) 간의 직렬 통신 인터페이스를 구현하여 외부 아날로그 신호를 디지털 데이터로 수집하는 것이다. 사용자가 선택한 채널 번호(add_sel)에 따라 ADC 변환 명령을 생성하고, SPI와 유사한 직렬 통신 방식으로 16비트 프레임을 송수신하여 변환 결과를 획득한다. 수신된 데이터의 하위 12비트를 실제 샘플값으로 저장하고, 상위 8비트는 LED로 출력하여 변환 결과를 확인할 수 있도록 설계하였다. 이를 통해 FPGA의 직렬 통신 제어, 데이터 수집 및 저장, ADC 인터페이스 설계 과정을 학습하는 것이 목표이다.

· **핵심 코드**

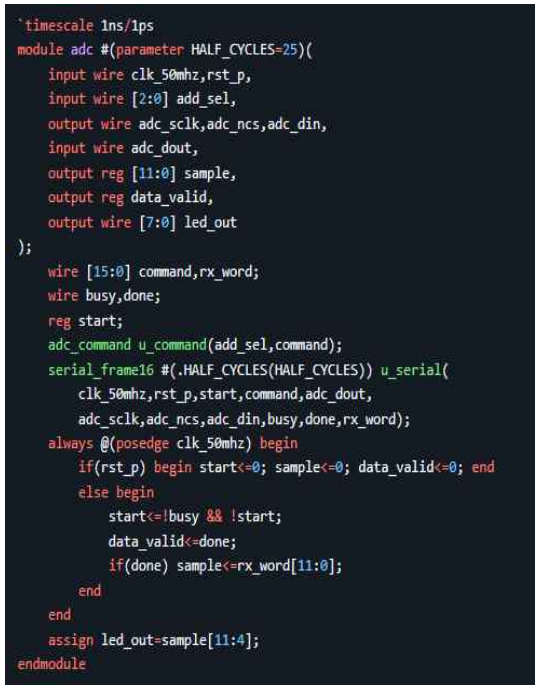
```
`timescale 1ns/1ps
module adc_command(input wire [2:0] channel,output wire [15:0] command);
    // 원본 예제의 WR·SEQ·주소·전원·범위·코딩 비트 구성을 유지합니다.
    assign command={3'b101,channel,2'b11,1'b0,3'b111,4'b0000};
endmodule
```

adc_command 모듈은 ADC에 전송할 16비트 제어 명령어를 생성하는 기능을 담당한다. 입력으로 3비트 채널 번호(channel)를 받아 ADC의 채널 선택 비트 영역에 삽입하고, 나머지 비트는 WR, SEQ, 전원 설정, 입력 범위, 데이터 포맷 등의 제어 정보로 채워진다. 생성된 명령어는 이후 직렬 통신 모듈을 통해 ADC로 전송된다. 즉, 이 모듈은 사용자가 선택한 ADC 입력 채널을 ADC 장치가 인식할 수 있는 제어 프레임 형태로 변환하는 역할을 수행한다.



```
`timescale 1ns/1ps
// 16비트 직렬 송수신. 클록은 HIGH 대기, 하강 에지에서 준비, 상승 에지에서 수신합니다.
module serial_frame16 #(parameter HALF_CYCLES=25)(
    input wire clk_50mhz,rst_p,start,
    input wire [15:0] tx_word,
    input wire miso,
    output reg sclk,cs_n,mosi,busy,done,
    output reg [15:0] rx_word
);
    reg [31:0] count;
    reg [4:0] bit_index;
    reg [15:0] tx_latched;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin
            sclk<=1; cs_n<=1; mosi<=0; busy<=0; done<=0;
            count<=0; bit_index<=15; tx_latched<=0; rx_word<=0;
        end else begin
            done<=0;
            if(!busy) begin
                if(start) begin
                    cs_n<=0; busy<=1; count<=0; bit_index<=15;
                    tx_latched<=tx_word; rx_word<=0; mosi<=tx_word[15];
                end
            end else if(count==HALF_CYCLES-1) begin
                count<=0;
                if(sclk) begin sclk<=0; mosi<=tx_latched[bit_index]; end
                else begin
                    sclk<=1;
                    rx_word<={rx_word[14:0],miso};
                    if(bit_index==0) begin busy<=0; cs_n<=1; done<=1; end
                    else bit_index<=bit_index-1'b1;
                end
            end else count<=count+1;
        end
    end
endmodule
```

serial_frame16 모듈은 ADC와의 실제 16비트 직렬 송수신을 수행하는 핵심 통신 회로이다. start 신호가 입력되면 송신 데이터(tx_word)를 내부 레지스터에 저장한 후, 16비트 프레임 동안 CS를 활성화하고 SCLK를 생성한다. 클록 하강 에지에서는 송신 비트(MOSI)를 준비하고, 상승 에지에서는 ADC가 반환하는 수신 비트(MISO)를 읽어 rx_word에 저장한다. 16비트 수신이 완료되면 done 신호를 발생시켜 상위 모듈에 데이터 수신 완료를 알린다. 이를 통해 ADC와 FPGA 간의 안정적인 데이터 교환이 이루어진다.



```
`timescale 1ns/1ps
module adc #(parameter HALF_CYCLES=25)(
    input wire clk_50mhz,rst_p,
    input wire [2:0] add_sel,
    output wire adc_sclk,adc_ncs,adc_din,
    input wire adc_dout,
    output reg [11:0] sample,
    output reg data_valid,
    output wire [7:0] led_out
);
    wire [15:0] command,rx_word;
    wire busy,done;
    reg start;
    adc_command u_command(add_sel,command);
    serial_frame16 #(.HALF_CYCLES(HALF_CYCLES)) u_serial(
        clk_50mhz,rst_p,start,command,adc_dout,
        adc_sclk,adc_ncs,adc_din,busy,done,rx_word);
    always @(posedge clk_50mhz) begin
        if(rst_p) begin start<=0; sample<=0; data_valid<=0; end
        else begin
            start<=!busy && !start;
            data_valid<=done;
            if(done) sample<=rx_word[11:0];
        end
    end
    assign led_out=sample[11:4];
endmodule
```

adc 모듈은 ADC 시스템 전체를 제어하는 최상위 연결 모듈이다. 내부적으로 adc_command와 serial_frame16 모듈을 연결하여 채널 선택, 명령 전송, 데이터 수신 기능을 통합한다. 통신이 완료되면 수신 데이터(rx_word)의 하위 12비트를 sample 레지스터에 저장하고, data_valid 신호를 통해 새 데이터 수신 여부를 알린다. 또한 sample[11:4] 값을 LED 출력으로 연결하여 변환 결과를 시각적으로 확인할 수 있도록 구성하였다. 결과적으로 이 모듈은 ADC 제어, 데이터 저장, 출력 표시 기능을 하나의 시스템으로 통합하는 역할을 수행한다.

```
`timescale 1ns/1ps
module tb;
    reg clk=0,rst=1;
    always #5 clk=~clk;
    reg [2:0] channel=2;
    reg miso=0;
    wire sclk,cs_n,mosi,valid;
    wire [11:0] sample;
    wire [7:0] led;
    adc #(.HALF_CYCLES(2)) dut(clk,rst,channel,sclk,cs_n,mosi,miso,sample,valid,led);
    reg [15:0] received_command;
    reg [15:0] reply=16'hA5C3;
    reg [2:0] frame_channel;
    integer bit_no,received_bits,frames=0;
    // ADC 모델: 하강 에지에서 응답 비트 준비, 상승 에지에서 명령 수신.
    always @(negedge cs_n) begin
        bit_no=15; received_bits=0; received_command=0; frame_channel=channel;
    end
    always @(negedge sclk)
        if(!cs_n) miso=reply[bit_no];
    always @(posedge sclk)
        if(!rst && received_bits<16) begin
            received_command={received_command[14:0],mosi};
            received_bits=received_bits+1;
            bit_no=bit_no-1;
        end
    always @(posedge valid) begin
        #1;
        if(received_bits != 16) $fatal(1,"ADC frame length");
        if(received_command !== (16'hA370 | (frame_channel << 10)))
            $fatal(1,"ADC command channel=%0d cmd=%h",frame_channel,received_command);
        if(sample !== reply[11:0] || led !== reply[11:4]) $fatal(1,"ADC sample");
        frames=frames+1;
    end
    initial begin
        repeat(2) @(negedge clk);
        if(cs_n!==1 || sclk!==1 || sample!==0) $fatal(1,"ADC reset");
        rst=0;
        wait(frames==1); @(negedge clk); channel=5;
        wait(frames==2); @(negedge clk); channel=7;
        wait(frames==3); @(negedge clk); rst=1;
        @(posedge clk); #1;
        if(cs_n!==1 || sclk!==1 || sample!==0 || valid!==0) $fatal(1,"ADC reset after data");
        $display("PASS ADC: 3 serial frames, command bits, sample, reset"); $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

TB는 ADC 장치 대신 간단한 ADC 모델을 구성하여 FPGA 설계가 정상적으로 동작하는지 자동 검증한다. 시뮬레이션에서는 ADC 응답값으로 16'hA5C3을 설정하고, FPGA가 생성한 명령어 비트를 수신하여 올바른 채널 정보가 포함되었는지 검사한다. 또한 세 번의 직렬 프레임 전송 과정에서 채널 값을 2, 5, 7로 변경하면서 명령어 생성이 정상적으로 이루어지는지 확인한다. 수신 완료 후에는 샘플 데이터가 응답값의 하위 12비트(5C3)와 일치하는지, LED 출력이 상위 8비트(5C)와 일치하는지 검증한다. 마지막으로 리셋 입력 시 모든 출력이 초기 상태로 복원되는지 확인하여 ADC 인터페이스 전체 기능의 정상 동작을 검사한다.

· 시뮬레이션 자료

- Bit Hash

```
PS D:\Engineering\Vivado_Work\Lab5> Get-FileHash -LiteralPath 'D:\Engineering\Vivado_Work\Lab5\lab5_adc\lab5_adc.runs\impl_1\adc.bit' -Algorithm SHA256 | Format-List Path,Hash


Path : D:\Engineering\Vivado_Work\Lab5\lab5_adc\lab5_adc.runs\impl_1\adc.bit
Hash : B90FA0BAC3E92C5D81DC4C536E64A0A777F683603746ZB8F683133CF3614E018
```

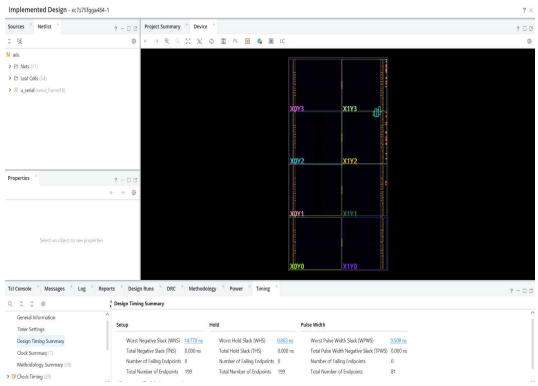
- Bitstream



- DRC

Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax:

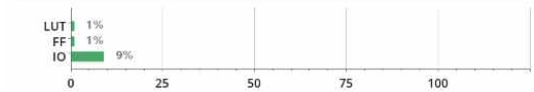
- Timing



- Utilization

Summary

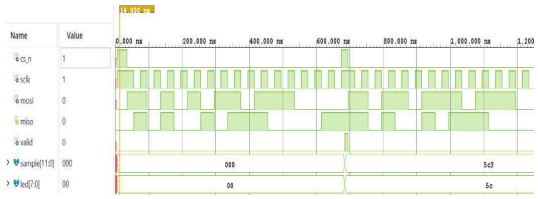
Resource	Utilization	Available	Utilization %
LUT	57	48000	0.12
FF	80	96000	0.08
IO	30	338	8.88



- Vivado Pass

```
run all
PASS ADC: 3 serial frames, command bits, sample, reset
$finish called at time : 2016 ns : File "D:/Engineering/Verilog_Lab/ece2-lab5/fpga/adc/tb.sv" Line 69
```

- Simulation



· **검증 결과**

시뮬레이션 결과 3개의 직렬 통신 프레임이 모두 정상적으로 수행되었다. 각 프레임에서 ADC 명령어의 비트 구성이 설계한 채널 값에 맞게 생성되었으며, 16비트 송수신 과정에서도 데이터 손실 없이 정확하게 전달되었다. ADC 모델이 반환한 응답값의 하위 12비트가 sample 레지스터에 정상 저장되었고, LED 출력 역시 sample[11:4] 값과 일치하였다. 또한 리셋 신호 입력 시 sample, data_valid, CS, SCLK가 모두 초기 상태로 복원됨을 확인하였다. 최종적으로 시뮬레이션에서는 "PASS ADC: 3 serial frames, command bits, sample, reset" 메시지가 출력되어 설계가 의도한 대로 동작함을 검증하였다.

· **문제 해결 과정**

ADC 설계 과정에서 가장 중요한 문제는 FPGA와 ADC 사이의 직렬 통신 타이밍을 정확히 맞추는 것이었다. 데이터 송신과 수신이 동일한 클록에서 이루어지면 비트가 어긋날 수 있으므로, 하강 에지에서 송신 데이터를 준비하고 상승 에지에서 수신 데이터를 읽는 구조를 적용하여 안정적인 통신을 구현하였다. 또한 ADC 변환 결과를 저장할 때 이전 데이터와 새로운 데이터가 충돌하지 않도록 busy, start, done 신호를 이용한 프레임 제어 방식을 적용하였다. 채널 변경 시에는 새로운 명령어가 다음 프레임부터 반영되도록 설계하여 통신 중 데이터 오류를 방지하였다. 마지막으로 LED 출력을 추가하여 수신 데이터를 즉시 확인할 수 있도록 함으로써 디버깅과 검증 과정을 효율적으로 수행할 수 있었다.

4. STM32_1 : 과제 3 - 인터럽트

· **설계 목표**

STM32_0 과제1에서 구현한 버튼 기반 LED 점멸 주기 변경 기능을 인터럽트 방식으로 재구현하는 것을 목표로 하였다. 버튼을 누를 때마다 LED 점멸 간격이 1초 → 2초 → 4초 → 8초 → 1초 순으로 변경되도록 설계하였으며, 메인 루프의 폴링 방식 대신 버튼 인터럽트와 TIM6 타이머 인터럽트를 이용하여 더욱 효율적인 이벤트 기반 제어를 구현하였다.

· **설계 방법**

버튼 입력은 button_isr_p7() 인터럽트 서비스 루틴에서 처리하도록 하였다. 버튼이 눌릴 때마다 LED 점멸 간격을 저장하는 변수를 변경하여 1초, 2초, 4초, 8초의 순서로 순환하도록 구현하였다. LED 제어는 tim6_isr_p4()에서 수행하였으며, TIM6이 100ms마다 발생시키는 인터럽트 횟수를 카운트하여 현재 설정된 점멸 주기에 도달하면 LED 상태를 반전시켰다. 또한 ece2_474_init() 함수에서 TIM6 인터럽트를 활성화하고, 기존에 메인 루프에서 수행하던 버튼 및 LED 제어 코드는 제거하여 모든 동작이 ISR 내부에서 이루어지도록 구성하였다.

· **회로 구조**

회로는 STM32 보드의 사용자 버튼과 내장 LED(PA5), 그리고 TIM6 타이머를 중심으로 구성하였다. 버튼 입력은 외부 인터럽트를 통해 MCU로 전달되며, 인터럽트 발생 시 점멸 주기 변수가 변경된다. TIM6은 100ms 주기로 인터럽트를 발생시키며, 인터럽트 핸들러 내부에서 카운터 값을 누적하여 설정된 시간에 도달하면 내장 LED를 토글한다. 따라서 버튼 인터럽트는 점멸 주기를 결정하는 역할을 수행하고, 타이머 인터럽트는 실제 LED 점멸 동작을 담당하는 구조로 설계하였다.

· **핵심 코드**

```
#include "ece2_474.h"

volatile uint32_t led_interval = 10U;
```

led_interval 변수는 LED의 점멸 주기를 저장하기 위한 전역 변수이다. TIM6 인터럽트가 100ms마다 발생하므로, 해당 변수에는 LED 상태를 변경하기까지 필요한 인터럽트 횟수를 저장한다. 초기값인 10은 약 1초를 의미하며, 버튼 입력에 따라 20(2초), 40(4초), 80(8초)으로 변경된다. 전역 변수로 선언하여 버튼 인터럽트 함수와 타이머 인터럽트 함수가 동일한 값을 공유할 수 있도록 하였다.

```
HAL_TIM_Base_Start_IT(htim: &htim6);

    /* TIM3: PB4에서 1 kHz PWM을 출력합니다. 비교값 500은 50%입니다. */
HAL_TIM_PWM_Start(htim: &htim3, Channel: TIM_CHANNEL_1);
HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 500U);
```

HAL_TIM_Base_Start_IT() 함수는 TIM6 타이머를 인터럽트 모드로 동작시키기 위해 사용하였다. TIM6은 100ms마다 인터럽트를 발생시키며, 인터럽트 발생 시 tim6_isr_p4() 함수가 호출된다. 이를 통해 메인 루프에서 지속적으로 시간을 확인하는 폴링 방식 대신 인터럽트 기반의 주기 제어를 구현하여 시스템 자원을 효율적으로 사용할 수 있다.

```
static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();

    if(now_ms - last_ms >= 200U)
    {
        last_ms = now_ms;

        switch(led_interval)
        {
        case 10U:
            led_interval = 20U;
            break;

        case 20U:
            led_interval = 40U;
            break;

        case 40U:
            led_interval = 80U;
            break;

        default:
            led_interval = 10U;
            break;
        }
    }
}
```

button_isr_p7() 함수는 사용자 버튼이 눌렸을 때 실행되는 인터럽트 서비스 루틴이다. 함수 내부에서는 HAL_GetTick()을 이용하여 버튼 입력 간의 시간을 측정하고, 200ms 이내에 발생하는 중복 입력을 무시하는 디바운싱 기능을 수행한다. 또한 버튼이 정상적으로 입력되면 led_interval 값을 10, 20, 40, 80 순으로 변경하여 LED 점멸 주기를 단계적으로 증가시키고, 마지막 단계에서는 다시 초기값으로 복귀하도록 구현하였다.

```
static void tim6_isr_p4(void)
{
    static uint32_t count = 0;

    if(++count >= led_interval)
    {
        count = 0;

        HAL_GPIO_TogglePin(
            GPIOx: LED_ece2_GPIO_Port,
            GPIO_Pin: LED_ece2_Pin
        );
    }
}
```

tim6_isr_p4() 함수는 TIM6 인터럽트가 발

생할 때마다 호출되는 함수이다. 내부 카운터 변수를 이용하여 인터럽트 발생 횟수를 누적하고, 누적 횟수가 led_interval 값에 도달하면 LED의 상태를 반전시킨다. 이를 통해 버튼 인터럽트에서 설정된 주기에 맞추어 LED가 1초, 2초, 4초, 8초 간격으로 점멸하도록 구현하였다. 또한 인터럽트 내부에서는 짧은 작업만 수행하여 실시간 제어 시스템의 효율성을 유지하도록 설계하였다.

· 검증 결과

프로그램 다운로드 후 실험한 결과, LED는 기본적으로 1초 주기로 점멸하였으며 버튼을 한 번 누를 때마다 점멸 주기가 2초, 4초, 8초로 순차적으로 변경되었다. 이후 다시 버튼을 누르면 1초 주기로 복귀하며 동일한 순환 동작이 정상적으로 반복되었다. 또한 버튼을 길게 누르더라도 인터럽트가 한 번만 처리되어 의도한 대로 점멸 간격이 한 단계씩만 변경되는 것을 확인하였다. 이를 통해 인터럽트 기반 LED 제어와 주기 변경 기능이 정상적으로 구현되었음을 검증하였다.

· 문제 해결 과정

실험 초기에는 버튼을 한 번 눌렀을 때 점멸 주기가 여러 단계 변경되는 현상이 발생하였다. 이는 스위치의 채터링 현상과 인터럽트의 중복 발생 때문이었다. 이를 해결하기 위해 버튼 인터럽트 내에서 디바운싱 기법을 적용하여 일정 시간 내의 중복 입력을 무시하도록 하였다. 또한 ISR 간에 공유되는 변수는 전역 변수로 선언하고 volatile 키워드를 사용하여 컴파일러 최적화에 의한 오동작을 방지하였다. 이후 실험에서 안정적으로 점멸 주기가 변경되는 것을 확인하였다.

5. STM32_1 : 과제 4 - PWM

· 설계 목표

버튼 입력에 따라 PWM 듀티비를 25% → 50% → 75% → 25% 순으로 변경하는 기능을 구현하는 것을 목표로 하였다. TIM3의 PWM 출력을 이용하여 듀티비 변화를 생성하고, 버튼 인터럽트를 사용하여 사용자의 입력에 따라 PWM 비율이 순환적으로 변하도록 설계하였다. 또한 C 디버거를 이용하여 변수 값과 PWM 설정값의 변화를 직접 확인함으로써 디버깅 방법을 익히는 것을 목표로 하였다.

· 설계 방법

초기화 함수인 ece2_474_init()에서 HAL_TIM_PWM_Start()를 사용하여 TIM3 PWM 출력을 시작하였다. 버튼 입력은 button_isr_p7()에서 처리하였으며, 버튼이 눌릴 때마다 PWM 듀티비 상태 변수를 변경하였다.

이후 __HAL_TIM_SET_COMPARE() 매크로를 사용하여 비교 레지스터 값을 각각 250, 500, 750으로 변경함으로써 25%, 50%, 75% 듀티비를 생성하였다. 또한 디버깅 과정에서는 버튼 인터럽트 내부에 브레이크포인트를 설정하고 Variables 및 Watch 창을 이용하여 PWM 상태 변수와 비교 레지스터 값의 변화를 확인하였다.

· 회로 구조

회로는 STM32의 사용자 버튼, TIM3 PWM 출력 핀(PB4), 그리고 측정 장비(오실로스코프 또는 로직 애널라이저)로 구성하였다. 버튼 입력이 인터럽트로 MCU에 전달되면 PWM 듀티비 설정값이 변경되며, TIM3은

이에 따라 새로운 PWM 파형을 출력한다. PB4 출력은 측정 장비에 연결하여 듀티비 변화를 관찰할 수 있도록 하였다. 버튼 인터럽트는 PWM 비율 선택 기능을 담당하고, TIM3 PWM 모듈은 실제 PWM 신호를 생성하는 역할을 수행하였다.

· **핵심 코드**

```c
#include "ece2_474.h"

#include "main.h"

volatile uint32_t pwm_compare = 250U;
```

pwm_compare 변수는 PWM 신호의 듀티비를 결정하는 비교값(CCR 값)을 저장하기 위한 전역 변수이다. TIM3의 주기를 1000으로 설정한 상태에서 250은 25%, 500은 50%, 750은 75%의 듀티비를 의미한다. 버튼 인터럽트와 PWM 출력 설정 함수가 동일한 값을 사용해야 하므로 전역 변수로 선언하였으며, 버튼 입력에 따라 PWM 출력 비율이 변경될 수 있도록 하였다.

```c
void ece2_474_init(void)
{
    /* 필요한 기능만 주석을 해제합니다. 기본 핀·통신 설정은 이미 끝난 상태입니다.
     * VS Code에서 해당 줄에 커서를 놓거나 여러 줄을 선택하고 Ctrl + /를 누르면
     * 줄 주석(//)을 해제하거나 다시 설정할 수 있습니다.
     */

    /* TIM6: 100 ms마다 tim6_isr_p4()를 호출합니다. */
    // HAL_TIM_Base_Start_IT(&htim6);

    /* TIM3: PB4에서 1 kHz PWM을 출력합니다. 비교값 500은 50%입니다. */
    HAL_TIM_PWM_Start(htim: &htim3, Channel: TIM_CHANNEL_1);

    __HAL_TIM_SET_COMPARE(
        &htim3,
        TIM_CHANNEL_1,
        pwm_compare
    );
```

HAL_TIM_PWM_Start() 함수는 TIM3의 PWM 출력을 시작하기 위해 사용하였다. 또한 __HAL_TIM_SET_COMPARE() 함수는 타이머의 비교 레지스터(CCR) 값을 설정하여 듀티비를 결정한다. 초기값으로 pwm_compare를 적용하여 실험 시작 시 25% 듀티비의 PWM 신호가 출력되도록 하였으며, 이후 버튼 입력에 따라 변경되는 값을 즉시 PWM 출력에 반영할 수 있도록 구성하였다.

```c
static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();

    if(now_ms - last_ms >= 200U)
    {
        last_ms = now_ms;

        switch(pwm_compare)
        {
        case 250U:
            pwm_compare = 500U;
            break;

        case 500U:
            pwm_compare = 750U;
            break;

        default:
            pwm_compare = 250U;
            break;
        }

        __HAL_TIM_SET_COMPARE(
            &htim3,
            TIM_CHANNEL_1,
            pwm_compare
        );
    }
}
```

button_isr_p7() 함수는 사용자 버튼이 눌렸을 때 실행되는 인터럽트 서비스 루틴이다. 함수 내부에서는 HAL_GetTick()을 이용한 디바운싱 처리를 수행하여 스위치 접점 잡음으로 인한 오동작을 방지하였다. 버튼이 정상적으로 입력되면 pwm_compare 값을 250, 500, 750 순으로 변경하고, 마지막 단계 이후에는 다시 250으로 초기화한다. 또한 변경된 값을 __HAL_TIM_SET_COMPARE() 함수에 적용하여 PWM 듀티비가 즉시 갱신되도록 구현하였다.

```c
static void tim6_isr_p4(void)
{
    static uint32_t count = 0;

    if(++count >= led_interval)
    {
        count = 0;

        HAL_GPIO_TogglePin(
            GPIOx: LED_ece2_GPIO_Port,
            GPIO_Pin: LED_ece2_Pin
        );
    }
}
```

tim6_isr_p4() 함수는 TIM6 인터럽트가 발생할 때마다 호출되는 함수이다. 내부 카운터 변수를 이용하여 인터럽트 발생 횟수를 누적하고, 누적 횟수가 led_interval 값에

도달하면 LED의 상태를 반전시킨다. 이를 통해 버튼 인터럽트에서 설정된 주기에 맞추어 LED가 1초, 2초, 4초, 8초 간격으로 점멸하도록 구현하였다. 또한 인터럽트 내부에서는 짧은 작업만 수행하여 실시간 제어 시스템의 효율성을 유지하도록 설계하였다.

· 검증 결과

실험 결과 PWM 출력은 초기 25% 듀티비에서 시작하였으며 버튼을 누를 때마다 50%, 75%, 다시 25%로 정상적으로 변경되었다. 오실로스코프를 통해 PWM 파형을 측정한 결과 High 구간의 비율이 각각 전체 주기의 약 25%, 50%, 75%가 되는 것을 확인하였다. 또한 디버거를 이용하여 상태 변수와 비교 레지스터 값이 250, 500, 750으로 순서대로 변경되는 것을 확인하였으며, PWM 출력 역시 이에 맞게 정상적으로 변화하였다.

· 문제 해결 과정

초기 구현에서는 PWM 듀티비가 변경되지 않거나 예상과 다른 비율로 출력되는 문제가 발생하였다. 원인을 분석한 결과 PWM 채널 시작 코드가 누락되었거나 비교 레지스터 값 갱신이 정상적으로 수행되지 않은 경우가 있음을 확인하였다. 이를 해결하기 위해 HAL_TIM_PWM_Start()가 초기화 과정에서 정상적으로 호출되는지 확인하고, 버튼 인터럽트마다 __HAL_TIM_SET_COMPARE()가 실행되도록 코드를 수정하였다. 또한 디버거를 이용해 변수값과 레지스터 값을 단계별로 추적하여 PWM 설정이 올바르게 반영되는 것을 검증하였고, 최종적으로 모든 듀티비가 정상적으로 출력되는 것을 확인하였다.

6. 최종 고찰

과제 3에서는 인터럽트와 타이머를 활용한 이벤트 기반 프로그래밍 기법을 학습할 수 있었다. 기존의 폴링 방식과 비교하여 버튼 입력이 발생했을 때만 처리하도록 구현함으로써 시스템 자원을 보다 효율적으로 사용할 수 있음을 확인하였다. 또한 TIM6 타이머 인터럽트를 이용하여 LED 점멸 주기를 정확하게 제어하고, 버튼 입력에 따라 1초, 2초, 4초, 8초로 점멸 간격이 변경되도록 구현하면서 인터럽트 서비스 루틴(ISR)의 동작 원리와 타이머 활용 방법을 이해할 수 있었다. 실험 과정에서는 스위치 채터링으로 인한 오동작 문제를 경험하였으며, 이를 디바운싱 기법을 통해 해결하면서 실제 임베디드 시스템 설계 시 고려해야 할 요소를 학습할 수 있었다.

과제 4에서는 STM32의 PWM 기능을 이용하여 듀티비를 제어하고 출력 파형의 변화를 확인함으로써 하드웨어 타이머의 활용 방법을 익힐 수 있었다. 버튼 인터럽트를 통해 PWM 듀티비를 25%, 50%, 75%로 변경하고 오실로스코프 및 디버거를 통해 출력 결과를 검증하면서 비교 레지스터 값과 PWM 파형의 관계를 명확히 이해할 수 있었다. 또한 디버깅 기능을 활용하여 변수와 레지스터 값의 변화를 추적함으로써 프로그램의 동작 과정을 분석하는 능력을 향상시킬 수 있었다. 이번 실험을 통해 PWM이 LED 밝기 제어, 모터 속도 제어 등 다양한 임베디드 응용 분야에서 중요한 역할을 수행한다는 것을 확인하였으며, STM32의 타이머 및 인터럽트 기능에 대한 이해를 더욱 심화할 수 있었다.

7. 참고 문헌

[1] STMicroelectronics, STM32F4 Series Reference Manual (RM0090).
[2] STMicroelectronics, STM32Cube HAL Driver User Manual.
[3] 서울시립대학교 전자전기컴퓨터공학부, STM32_1 실험자료

8. Git-Hub 주소 및 최종 커밋

https://github.com/UnnamedDays/ECE-II-Project

4fcaed7